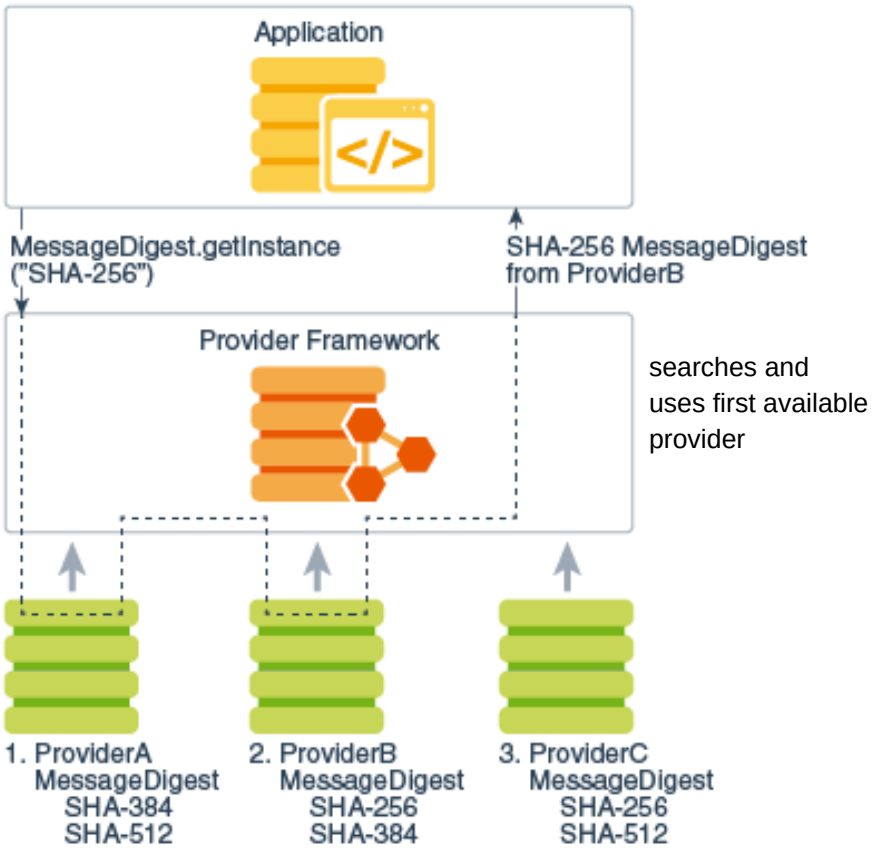
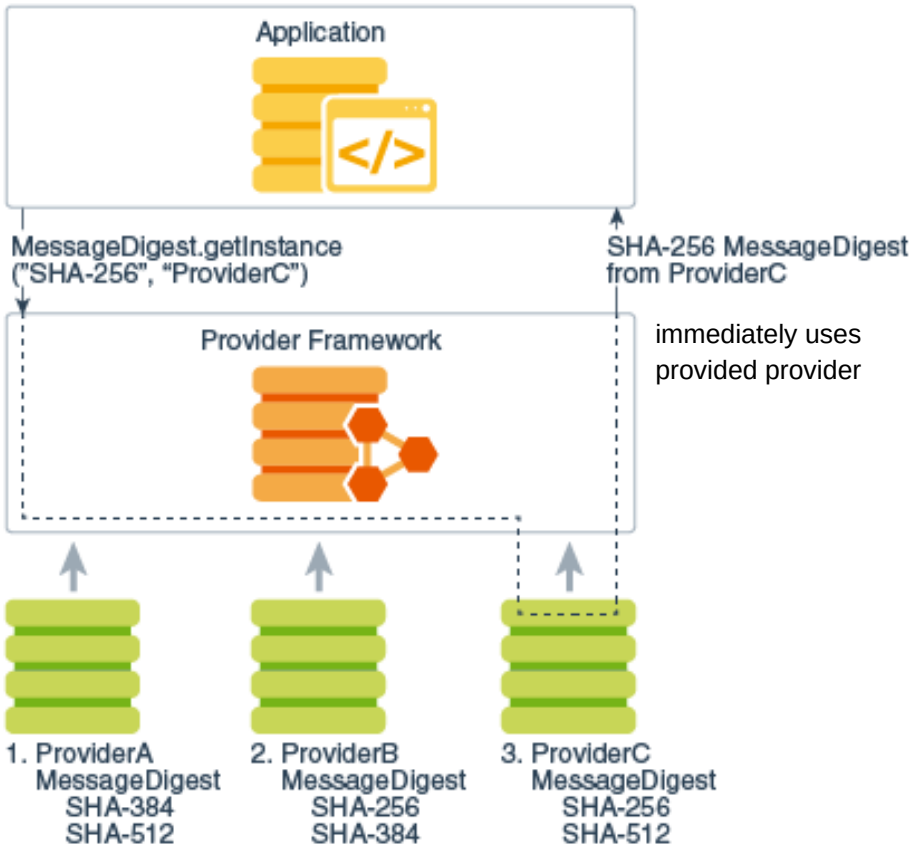
Basic security architecture of Java

- Implementation Independence
  - Security is implemented in providers that are addressed via a standard interface
  - Several providers can be configured per application (controlled by priorities)
- Implementation interoperability
  - Providers are interoperable
- Algorithm extensibility
  - The services of the providers can be extended
- Accessing a provider's security service via getInstance method and factory pattern
  - MessageDigest md = MessageDigest.getInstance(“SHA-256");
  - MessageDigest md = MessageDigest.getInstance(“SHA-256", "ProviderC");

Security Providers



Request SHA-256 Message Digest Implementation
without specifying provider



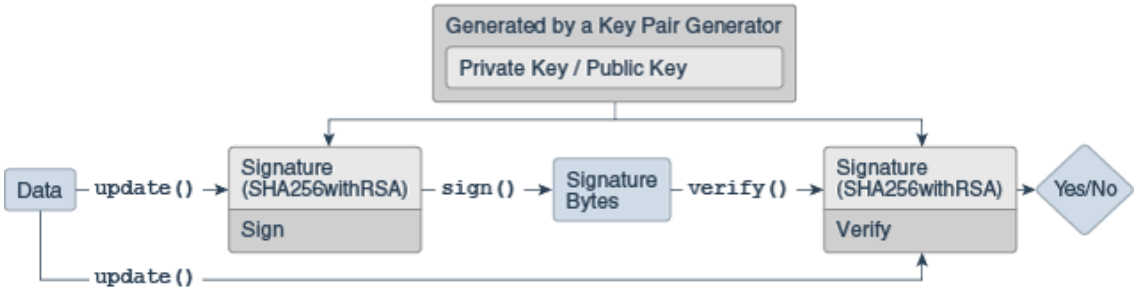
Request SHA-256 Message Digest Implementation with ProviderC

JCE Java Cryptography Extension (API for cryptographic services)

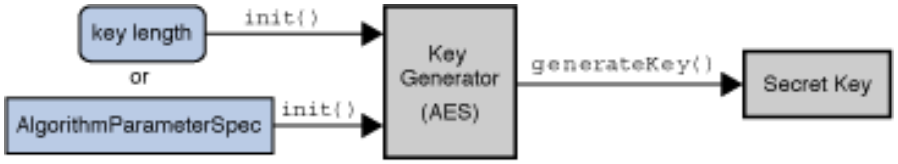
- Message digest algorithms
- Digital signature algorithms
- Symmetric bulk encryption
- Symmetric stream encryption
- Asymmetric encryption
- Password-based encryption (PBE)
- Elliptic Curve Cryptography (ECC)
- Key agreement algorithms
- Key generators
- Message Authentication Codes (MACs)
- (Pseudo-)random number generators

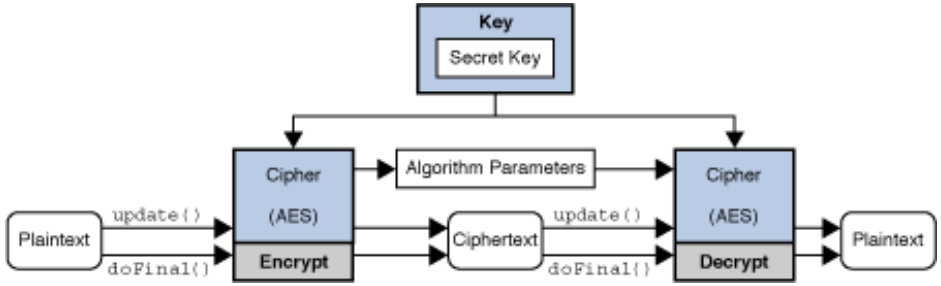
Package Structure of the Cryptographic Services API

- java.security
  - no export restrictions
  - e.g. Signature, MessageDigest



- javax.crypto
  - export Control
  - e.g. Cipher, KeyGenerator
  - providers must be digitally signed





- The JDK contains only a few common cryptographic algorithms (e.g. AES, RSA, SHA-256)

Notes on the use of JCE

- Documentation
  - https://docs.oracle.com/en/java/javase/23/security/java-cryptography-architecture-jca-reference-guide.html
  - https://docs.oracle.com/en/java/javase/23/docs/api/java.base/javax/crypto/package-summary.html
  - https://docs.oracle.com/en/java/javase/23/
- Principle of generators and factories
  - there are no constructors for most classes
  - to get instances of classes you must use the factory method getInstance()
  - Example:

```
Cipher c = Cipher.getInstance("AES");
c.init(ENCRYPT_MODE, key);
```